valid heaps.

We can't say yet if `X` is the root of a valid heap: but we can try to push it down and possibly swap it with the smallest of its children. Once we have done that, we are sure that the sub-heap rooted at `X` is a valid heap. This happens because `pushDown`, by construction, reinstates the heap properties on a sub-heap where all children are valid heaps, and only the root might be misplaced.

If we now move one position left in the array, we can repeat the procedure, and get another valid sub-heap. Keeping on, at some point we visit all internal nodes that only have leaves as their children. And after that, we'll start with the first internal node that's the root of a sub-heap with height 2, let's name it `Y`: if we tackle elements in a right-to-left order in the array, we have the guarantee that `Y`'s children are sub-heaps that

1. have at most height 1 and
2. whose properties have already been fixed.

So we can, once again, use `pushDown` and be sure the sub-heap rooted at `Y` will be a valid heap, afterwards.

If we repeat these steps for all nodes, till we reach the root of our heap, we have the guarantee that we will end up with a valid heap.

Listing 2.9 The heapify method

```
function heapify(pairs)
```